

Guía de Implementación

Sentinel AcademIA – Paso a Paso

Backend Python + Frontend Vue 3 + AWS + Oracle OCI

Equipo de Desarrollo

20 de junio de 2026

Sobre esta guía

Esta es una guía **paso a paso** para implementar Sentinel AcademIA. Está escrita asumiendo que **NO tienes asistencia** de una IA, por lo que cada paso incluye:

- **Qué** estamos haciendo y por qué
- **Cómo** hacerlo (comandos exactos)
- **Output esperado** (qué deberías ver)
- **Problemas comunes** y cómo resolverlos

Tiempo estimado total: 14-18 horas de desarrollo concentrado.

Índice

1. Pre-requisitos	3
1.1. Cuentas necesarias	3
1.2. Herramientas a instalar localmente	3
2. FASE 0: Setup Inicial	4
2.1. 0.1 Configurar AWS CLI	4
2.1.1. Obtener credenciales de AWS Academy	4
2.1.2. Configurar el archivo de credenciales	4
2.1.3. Verificar acceso	5
2.2. 0.2 Configurar Oracle OCI CLI	5
2.2.1. Crear API key en OCI	5
2.2.2. Configurar OCI CLI	6
2.2.3. Crear Compartment y obtener OCIDs	6
2.2.4. Probar OCI Generative AI	7
2.3. 0.3 Clonar el repositorio	7
3. FASE 1: Backend Core (Python Lambdas)	7
3.1. 1.1 Setup del proyecto Python	8
3.1.1. Estructura de carpetas	8
3.1.2. Archivo de dependencias (requirements.txt)	8
3.1.3. pyproject.toml	8

3.1.4. Variables de entorno (.env)	9
3.2. 1.2 Shared Modules	10
3.2.1. Config	10
3.2.2. Logger	10
3.2.3. HTTP Response Helpers	11
3.2.4. Error Handler	12
3.2.5. Correlation ID	12
3.2.6. DynamoDB Client	12
3.2.7. Pydantic Schemas	13
3.3. 1.3 Lambda createQueja (la mas importante)	14
3.4. 1.4 SAM Template (template.yaml)	16
3.5. 1.5 Deploy del backend	19
4. FASE 2: Backend LLM (OCI Integration)	19
4.1. 2.1 Cliente OCI Generative AI	19
4.2. 2.2 LLM Router con Circuit Breaker	20
5. FASE 3: Frontend Vue 3	21
5.1. 3.1 Setup	21
5.2. 3.2 Deploy a Vercel	21
6. FASE 4: Archivos + Email	21
6.1. 4.1 S3 + Presigned URLs	21
6.2. 4.2 Email con SES	22
7. FASE 5: RAG (Opcional)	23
7.1. 5.1 Ingesta del reglamento	23
8. FASE 6: Testing	23
8.1. 6.1 Tests unitarios	23
9. FASE 7: Deploy Final	25
9.1. 7.1 Deploy completo	25
9.2. 7.2 Smoke tests	25
10. Apéndice A: Comandos útiles	25
11. Apéndice B: Troubleshooting	26
12. Apéndice C: Variables de entorno	27
13. Apéndice D: Costos esperados	27

1 Pre-requisitos

1.1 Cuentas necesarias

Cuenta	Costo	Para qué
AWS Academy	\$50 USD créditos (gratis)	Lambda, SQS, SNS, DynamoDB, S3, SES, CloudWatch
Oracle Cloud Academy	Créditos disponibles	OCI Generative AI (LLM), Document AI, Vault
Vercel	Free tier	Deploy del frontend Vue 3
GitHub	Free para public	Repositorio + CI/CD con GitHub Actions
OpenAI/Anthropic (opcional)	-	Backup para LLMs en caso de emergencia

1.2 Herramientas a instalar localmente

Listing 1: Instalación de herramientas en Linux (Fedora/RHEL)

```

1 # Actualizar sistema
2 sudo dnf update -y
3
4 # Python 3.12
5 sudo dnf install -y python3.12 python3.12-pip python3.12-devel
6
7 # Node.js 20
8 curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
9 sudo dnf install -y nodejs
10
11 # AWS CLI v2
12 curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "
    awscliv2.zip"
13 unzip awscliv2.zip
14 sudo ./aws/install
15
16 # SAM CLI
17 pip3.12 install --user aws-sam-cli
18
19 # OCI CLI
20 bash -c "$(curl -L https://raw.githubusercontent.com/oracle/oci-cli/
    master/scripts/install/install.sh)"
21
22 # Tectonic (compilador LaTeX)
23 curl -fsSL "https://github.com/tectonic-typesetting/tectonic/releases/
    download/tectonic%400.16.9/tectonic-0.16.9-x86_64-unknown-linux-musl.
    tar.xz" -o /tmp/tectonic.tar.xz
24 tar -xJf /tmp/tectonic.tar.xz -C /tmp
25 sudo mv /tmp/tectonic /usr/local/bin/
26
27 # Git, jq, make
28 sudo dnf install -y git jq make
29
30 # Verificar
31 python3.12 --version
32 node --version

```

```

33 aws --version
34 sam --version
35 oci --version
36 tectonic --version
37 git --version

```

Listing 2: Instalación en macOS con Homebrew

```

1 brew install python@3.12 node awscli aws-sam-cli oci-cli tectonic git jq
2 brew link --overwrite python@3.12

```

Listing 3: Instalación en Windows

```

1 # Descargar installers desde:
2 # - Python: https://www.python.org/downloads/
3 # - Node.js: https://nodejs.org/
4 # - AWS CLI: https://awscli.amazonaws.com/AWSCLIV2.msi
5 # - SAM CLI: https://github.com/aws/aws-sam-cli/releases
6 # - OCI CLI: https://docs.oracle.com/en-us/iaas/Content/API/SDKDocs/
  cliinstall.htm

```

2 FASE 0: Setup Inicial

2.1 0.1 Configurar AWS CLI

2.1.1 Obtener credenciales de AWS Academy

1. Ve a tu laboratorio en AWS Academy
2. Click en “AWS Details” (esquina superior derecha)
3. Click en “Show” junto a AWS CLI credentials
4. Copia los 3 valores:
 - AWS_ACCESS_KEY_ID
 - AWS_SECRET_ACCESS_KEY
 - AWS_SESSION_TOKEN (este caduca en 3-4 horas)

2.1.2 Configurar el archivo de credenciales

Nunca pongas credenciales en código o archivos del proyecto. Usa el archivo de credenciales de AWS CLI:

Listing 4: Configurar AWS CLI

```

1 mkdir -p ~/.aws
2
3 # Crear archivo de credenciales
4 cat > ~/.aws/credentials << 'EOF'
5 [default]
6 aws_access_key_id = TU_ACCESS_KEY_AQUI
7 aws_secret_access_key = TU_SECRET_KEY_AQUI
8 aws_session_token = TU_SESSION_TOKEN_AQUI
9 EOF
10
11 # Crear archivo de configuracion

```

```

12 cat > ~/.aws/config << 'EOF'
13 [default]
14 region = us-east-1
15 output = json
16 EOF
17
18 # Permisos correctos (importante)
19 chmod 600 ~/.aws/credentials ~/.aws/config

```

IMPORTANTE: Las credenciales de AWS Academy **caducan cada 3-4 horas**. Cuando caduquen, repite el proceso.

2.1.3 Verificar acceso

Listing 5: Verificar que las credenciales funcionan

```

1 # Identidad
2 aws sts get-caller-identity
3 # Output esperado:
4 # {
5 #     "UserId": "AROA...",
6 #     "Account": "123456789012",
7 #     "Arn": "arn:aws:sts::123456789012:assumed-role/LabRole/..."
8 # }
9
10 # Verificar region
11 aws configure get region
12 # Output: us-east-1
13
14 # Verificar que LabRole existe
15 aws iam get-role --role-name LabRole
16
17 # Crear bucket para SAM artifacts
18 BUCKET_NAME="sentinel-sam-artifacts-$(date +%s)"
19 aws s3 mb s3://$BUCKET_NAME --region us-east-1
20 echo "Bucket creado: $BUCKET_NAME"
21 # GUARDAR este nombre, lo necesitaras despues
22 echo $BUCKET_NAME > ~/.sentinel-sam-bucket

```

2.2 0.2 Configurar Oracle OCI CLI

2.2.1 Crear API key en OCI

1. Ve a OCI Console: <https://cloud.oracle.com/>
2. Click en tu perfil (esquina superior derecha) → **User Settings**
3. Seccion **API Keys** → Click “Add API Key”
4. Click “Download Private Key” y “Download Public Key”
5. Click “Add”
6. **Copia el Fingerprint** que aparece (lo necesitaras)

2.2.2 Configurar OCI CLI

Listing 6: Instalar y configurar OCI CLI

```

1 # Instalar
2 bash -c "$(curl -L https://raw.githubusercontent.com/oracle/oci-cli/
   master/scripts/install/install.sh)"
3
4 # Agregar al PATH (agregar a tu ~/.bashrc o ~/.zshrc)
5 echo 'export PATH="$HOME/lib/oracle-cli/bin:$PATH"' >> ~/.bashrc
6 source ~/.bashrc
7
8 # Verificar instalacion
9 oci --version

```

Listing 7: Configurar OCI con el config file

```

1 # Crear directorio de OCI
2 mkdir -p ~/.oci
3
4 # Mover las keys descargadas
5 mv ~/Downloads/api_key.pem ~/.oci/
6 mv ~/Downloads/*_fingerprint ~/.oci/ # el fingerprint file
7
8 # Configurar permisos
9 chmod 600 ~/.oci/api_key.pem
10
11 # Crear config file
12 cat > ~/.oci/config << EOF
13 [DEFAULT]
14 user=ocid1.user.oc1..TU_USER_OCID
15 tenancy=ocid1.tenancy.oc1..TU_TENANCY_OCID
16 region=us-chicago-1
17 key_file=$HOME/.oci/api_key.pem
18 fingerprint=TU_FINGERPRINT_AQUI
19 EOF
20
21 chmod 600 ~/.oci/config

```

Listing 8: Verificar acceso a OCI

```

1 # Test basico
2 oci iam region-subscription list
3 # Debe listar las regiones disponibles
4
5 # Obtener tu Tenancy OCID (si no lo tienes)
6 oci iam tenancy get --tenancy-id ocid1.tenancy.oc1..TU_TENANCY_OCID

```

2.2.3 Crear Compartment y obtener OCIDs

Listing 9: Setup de OCI

```

1 # Obtener Tenancy OCID
2 TENANCY_OCID=$(aws configure get tenancy_id 2>/dev/null || oci iam
   compartment list --all --query 'data[0].compartment-id' --raw-
   output 2>/dev/null)
3 # Si no funciona, sacalo del OCI Console
4

```

```

5 # Crear compartment para el proyecto
6 oci iam compartment create \
7   --compartment-id $TENANCY_OCID \
8   --name "SentinelAcademia" \
9   --description "Compartment para hackaton Sentinel Academia"
10
11 # GUARDAR el OCID del compartment que se creo
12 COMPARTMENT_OCID="ocid1.compartment.oc1..AQUI_TU_COMPARTMENT_OCID"
13 echo "export OCI_COMPARTMENT_ID=$COMPARTMENT_OCID" >> ~/.bashrc
14 echo $COMPARTMENT_OCID > ~/.sentinel-compartment

```

2.2.4 Probar OCI Generative AI

Listing 10: Test de OCI GenAI

```

1 # Verificar modelos disponibles
2 oci generative-ai-inference model list \
3   --compartment-id $COMPARTMENT_OCID \
4   --query 'data[*].display-name' \
5   --output table
6
7 # Probar Cohere Command R
8 oci generative-ai-inference generate-text \
9   --compartment-id $COMPARTMENT_OCID \
10  --serving-mode '{"modelId":"cohere.command-r","servingType":"ON_DEMAND"'
11  --inference-request '{"prompt":"Di hola en espanol","maxTokens":50}'
12
13 # Si funciona, veras un JSON con la respuesta

```

2.3 0.3 Clonar el repositorio

Listing 11: Setup del proyecto

```

1 # Clonar (o descargar como ZIP si es privado)
2 git clone https://github.com/tu-usuario/sentinel-academia.git
3 cd sentinel-academia
4
5 # Instalar dependencias del backend
6 cd src
7 python3.12 -m venv .venv
8 source .venv/bin/activate
9 pip install -r requirements.txt
10 cd ..
11
12 # Instalar dependencias del frontend
13 cd web
14 npm install
15 cd ..
16
17 # Verificar estructura
18 ls -la

```

3 FASE 1: Backend Core (Python Lambdas)

3.1 1.1 Setup del proyecto Python

3.1.1 Estructura de carpetas

Listing 12: Crear estructura del backend

```

1 cd src
2 mkdir -p handlers/_shared
3 mkdir -p services
4 mkdir -p types
5 mkdir -p tests/{fixtures,handlers,services}
6 touch handlers/__init__.py
7 touch handlers/_shared/__init__.py
8 touch services/__init__.py
9 touch types/__init__.py
10 touch tests/__init__.py

```

3.1.2 Archivo de dependencias (requirements.txt)

Listing 13: src/requirements.txt

```

1 boto3>=1.34
2 boto3-stubs[dynamodb,lambda,s3,sqs,sns,events]>=1.34
3 aws-lambda-powertools[tracer,metrics,validation]>=2.30
4 pydantic>=2.6
5 pydantic-settings>=2.2
6 opossum>=1.4
7 p-retry>=1.0
8 rate-limiter-flexible>=2.4
9 oci>=2.119
10
11 # Dev dependencies
12 pytest>=8.0
13 pytest-mock>=3.12
14 pytest-cov>=4.1
15 moto[dynamodb,s3,sqs,sns,events]>=5.0
16 freezegun>=1.4
17 responses>=0.25
18 ruff>=0.3
19 mypy>=1.8
20 black>=24.0
21 boto3-stubs[essential]>=1.34

```

3.1.3 pyproject.toml

Listing 14: src/pyproject.toml

```

1 [project]
2 name = "sentinel-academia-backend"
3 version = "0.1.0"
4 requires-python = ">=3.12"
5 dependencies = [
6     "boto3>=1.34",
7     "boto3-stubs[dynamodb,lambda,s3,sqs,sns]>=1.34",
8     "aws-lambda-powertools[tracer,metrics,validation]>=2.30",
9     "pydantic>=2.6",
10    "pydantic-settings>=2.2",

```



```

11     "opossum>=1.4",
12     "p-retry>=1.0",
13     "rate-limiter-flexible>=2.4",
14     "oci>=2.119",
15 ]
16
17 [project.optional-dependencies]
18 dev = [
19     "pytest>=8.0",
20     "pytest-mock>=3.12",
21     "pytest-cov>=4.1",
22     "moto[dynamodb,s3,sqs,sns,events]>=5.0",
23     "freezegun>=1.4",
24     "responses>=0.25",
25     "ruff>=0.3",
26     "mypy>=1.8",
27     "black>=24.0",
28 ]
29
30 [tool.ruff]
31 line-length = 100
32 target-version = "py312"
33
34 [tool.ruff.lint]
35 select = ["E", "F", "W", "I", "N", "UP", "B", "C4", "DTZ", "RET", "SIM"]
36
37 [tool.mypy]
38 python_version = "3.12"
39 strict = true
40 warn_unused_ignores = true
41 warn_return_any = true
42
43 [tool.pytest.ini_options]
44 testpaths = ["tests"]
45 addopts = "-v --cov=. --cov-report=term-missing"

```

3.1.4 Variables de entorno (.env)

Listing 15: src/.env

```

1 # AWS
2 AWS_REGION=us-east-1
3 DYNAMODB_TABLE=quejas
4 S3_BUCKET=sentinel-quejas-adjuntos-DONDE_VA_TU_ACCOUNT_ID
5 PROCESS_QUEUE_URL=https://sqs.us-east-1.amazonaws.com/
   DONDE_VA_TU_ACCOUNT_ID/complaints-queue
6 CRITICAL_ALERTS_TOPIC_ARN=arn:aws:sns:us-east-1:DONDE_VA_TU_ACCOUNT_ID:
   critical-alerts
7
8 # OCI
9 OCI_COMPARTMENT_ID=ocid1.compartment.oc1..AQUI_TU_COMPARTMENT
10 OCI_REGION=us-chicago-1
11 OCI_CONFIG_FILE=/home/tu-usuario/.oci/config
12 OCI_CONFIG_PROFILE=DEFAULT
13
14 # Email (SES)
15 SES_FROM_EMAIL=alerts@sentinel-academia.com

```

```

16 BIENESTAR_EMAIL=bienestar@universidad-ejemplo.edu
17 DIRECTOR_EMAIL=director@universidad-ejemplo.edu
18
19 # Multi-tenant
20 DEFAULT_TENANT_ID=universidad-ejemplo
21
22 # Logging
23 LOG_LEVEL=INFO
24 POWERTOOLS_SERVICE_NAME=sentinel-academia

```

3.2 1.2 Shared Modules

3.2.1 Config

Listing 16: src/handlers/_shared/config.py

```

1 """Configuracion tipada con pydantic-settings."""
2 from pydantic_settings import BaseSettings, SettingsConfigDict
3
4
5 class Settings(BaseSettings):
6     """Settings cargados desde variables de entorno."""
7
8     # AWS
9     aws_region: str = "us-east-1"
10    dynamodb_table: str = "quejas"
11    s3_bucket: str = "sentinel-quejas-adjuntos"
12    process_queue_url: str = ""
13    critical_alerts_topic_arn: str = ""
14
15    # OCI
16    oci_compartment_id: str = ""
17    oci_region: str = "us-chicago-1"
18
19    # Email
20    ses_from_email: str = "alerts@sentinel-academia.com"
21    bienestar_email: str = "bienestar@universidad.edu"
22    director_email: str = "director@universidad.edu"
23
24    # Multi-tenant
25    default_tenant_id: str = "universidad-ejemplo"
26
27    # Logging
28    log_level: str = "INFO"
29
30    model_config = SettingsConfigDict(
31        env_file=".env",
32        case_sensitive=False,
33        extra="ignore",
34    )
35
36
37 settings = Settings()

```

3.2.2 Logger

Listing 17: src/handlers/_shared/logger.py

```

1 """Logger con aws-lambda-powertools."""
2 from aws_lambda_powertools import Logger
3
4 logger = Logger(service="sentinel-academia")

```

3.2.3 HTTP Response Helpers

Listing 18: src/handlers/_shared/http_response.py

```

1 """Helpers para respuestas HTTP consistentes."""
2 import json
3 from typing import Any, Optional
4
5
6 def _response(status_code: int, body: Any, correlation_id: str) -> dict:
7     return {
8         "statusCode": status_code,
9         "headers": {
10             "Content-Type": "application/json",
11             "x-correlation-id": correlation_id,
12         },
13         "body": json.dumps(body, default=str),
14     }
15
16
17 def ok(body: Any, correlation_id: str) -> dict:
18     return _response(200, body, correlation_id)
19
20
21 def accepted(body: Any, correlation_id: str) -> dict:
22     return _response(202, body, correlation_id)
23
24
25 def bad_request(message: str, details: Optional[Any], correlation_id:
26 str) -> dict:
27     return _response(400, {
28         "code": "BAD_REQUEST", "message": message, "details": details, "
29         correlationId": correlation_id
30     }, correlation_id)
31
32
33 def not_found(message: str, correlation_id: str) -> dict:
34     return _response(404, {
35         "code": "NOT_FOUND", "message": message, "correlationId":
36         correlation_id
37     }, correlation_id)
38
39
40 def server_error(message: str, correlation_id: str) -> dict:
41     return _response(500, {
42         "code": "INTERNAL_ERROR", "message": message, "correlationId":
43         correlation_id
44     }, correlation_id)

```

3.2.4 Error Handler

Listing 19: src/handlers/_shared/errors.py

```

1  """Decorator para manejo uniforme de errores."""
2  from functools import wraps
3  from typing import Callable, Any
4  from pydantic import ValidationError
5  from .http_response import bad_request, server_error
6  from .logger import logger
7
8
9  def with_error_handling(handler_func: Callable) -> Callable:
10     @wraps(handler_func)
11     def wrapper(event: dict, context: Any) -> dict:
12         try:
13             return handler_func(event, context)
14         except ValidationError as e:
15             logger.warning("Validation error", extra={"errors": e.errors()})
16             return bad_request("Validation error", e.errors(),
17                                _get_correlation_id(event))
18         except Exception:
19             logger.exception("Unhandled error")
20             return server_error("Internal error", _get_correlation_id(
21                                 event))
22     return wrapper
23
24 def _get_correlation_id(event: dict) -> str:
25     return event.get("headers", {}).get("x-correlation-id", "unknown")

```

3.2.5 Correlation ID

Listing 20: src/handlers/_shared/correlation_id.py

```

1  """Helper para correlation ID."""
2  import uuid
3  from typing import Optional
4
5
6  def get_correlation_id(event: dict) -> str:
7     """Extrae correlation ID del header o genera uno nuevo."""
8     headers = event.get("headers", {}) or {}
9     return headers.get("x-correlation-id") or str(uuid.uuid4())

```

3.2.6 DynamoDB Client

Listing 21: src/handlers/_shared/dynamo_client.py

```

1  """Cliente DynamoDB cacheado."""
2  import os
3  from functools import lru_cache
4  from typing import Any
5  import boto3
6
7

```

```

8 @lru_cache(maxsize=1)
9 def get_dynamo_resource() -> Any:
10     return boto3.resource(
11         "dynamodb",
12         region_name=os.environ.get("AWS_REGION", "us-east-1"),
13     )
14
15 @lru_cache(maxsize=1)
16 def get_dynamo_client() -> Any:
17     return boto3.client(
18         "dynamodb",
19         region_name=os.environ.get("AWS_REGION", "us-east-1"),
20     )
21
22
23
24 TABLE_NAME = os.environ.get("DYNAMODB_TABLE", "quejas")
25 DEFAULT_TENANT_ID = os.environ.get("DEFAULT_TENANT_ID", "universidad-
    ejemplo")

```

3.2.7 Pydantic Schemas

Listing 22: src/handlers/_shared/schemas.py

```

1 """Schemas Pydantic para input/output."""
2 from enum import Enum
3 from typing import Optional, List
4 from pydantic import BaseModel, Field, field_validator, ConfigDict
5
6
7 class CategoriaEnum(str, Enum):
8     ACADEMICA = "ACADEMICA"
9     INFRAESTRUCTURA = "INFRAESTRUCTURA"
10    ACOSO = "ACOSO"
11    ADMINISTRATIVA = "ADMINISTRATIVA"
12    SALUD = "SALUD"
13    OTRA = "OTRA"
14
15
16 class CriticidadEnum(str, Enum):
17    BAJA = "BAJA"
18    MEDIA = "MEDIA"
19    ALTA = "ALTA"
20    CRITICA = "CRITICA"
21
22
23 class QuejaStatusEnum(str, Enum):
24    PENDIENTE = "PENDIENTE"
25    EN_COLA = "EN_COLA"
26    PROCESANDO = "PROCESANDO"
27    ANALIZADA = "ANALIZADA"
28    NOTIFICADA = "NOTIFICADA"
29    ERROR = "ERROR"
30
31
32 class CreateQuejaInput(BaseModel):
33    titulo: str = Field(min_length=5, max_length=120)

```

```

34     descripcion: str = Field(min_length=20, max_length=5000)
35     categoria_declarada: CategoriaEnum
36     adjuntos: Optional[List[str]] = Field(default=None, max_length=5)
37     anonima: bool = False
38     sede: Optional[str] = Field(default=None, max_length=100)
39     facultad: Optional[str] = Field(default=None, max_length=100)
40     contacto_email: Optional[str] = None
41
42     @field_validator("descripcion")
43     @classmethod
44     def descripcion_not_empty(cls, v: str) -> str:
45         if not v.strip():
46             raise ValueError("La descripcion no puede estar vacia")
47         return v.strip()
48
49     model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
50
51
52 class QuejaAccepted(BaseModel):
53     quejaId: str
54     status: QuejaStatusEnum
55     correlationId: str
56     createdAt: str
57     estimatedAnalysisTime: Optional[int] = None
58
59
60 class AnalisisOutput(BaseModel):
61     categoria: CategoriaEnum
62     subcategoria: str
63     criticidad: CriticidadEnum
64     criticidadJustificacion: str
65     sentimiento: str
66     entidades: List[dict] = Field(default_factory=list)
67     temasClave: List[str] = Field(default_factory=list)
68     accionSugerida: str
69     prioridad: int = Field(ge=1, le=10)
70     requiereNotificacionInmediata: bool
71     modeloUsado: str
72     tokensConsumidos: Optional[int] = None
73     latenciaMs: Optional[int] = None

```

3.3 1.3 Lambda createQueja (la mas importante)

Listing 23: src/handlers/create_queja.py

```

1  """POST /api/quejas - Crea una queja y la encola para analisis."""
2  import json
3  import uuid
4  from datetime import datetime, timezone
5  from typing import Any
6
7  import boto3
8  from aws_lambda_powertools import Logger
9  from pydantic import ValidationError
10
11 from ._shared.config import settings
12 from ._shared.correlation_id import get_correlation_id

```

```

13 from ._shared.dynamo_client import get_dynamo_resource
14 from ._shared.errors import with_error_handling
15 from ._shared.http_response import accepted, bad_request
16 from ._shared.schemas import CreateQuejaInput
17
18 logger = Logger(service="create-queja")
19
20
21 @logger.inject_lambda_context(log_event=True)
22 @with_error_handling
23 def handler(event: dict, context: Any) -> dict:
24     correlation_id = get_correlation_id(event)
25     logger.append_keys(correlation_id=correlation_id)
26     logger.info("create-queja invoked")
27
28     # 1. Validar input
29     try:
30         body = CreateQuejaInput.model_validate_json(event.get("body", "{}"))
31     except ValidationError as e:
32         return bad_request("Invalid input", e.errors(), correlation_id)
33
34     # 2. Generar IDs
35     queja_id = str(uuid.uuid4())
36     tenant_id = settings.default_tenant_id
37     logger.append_keys(queja_id=queja_id, tenant_id=tenant_id)
38
39     # 3. Persistir en DynamoDB
40     table = get_dynamo_resource().Table(settings.dynamodb_table)
41     now = datetime.now(timezone.utc)
42     table.put_item(Item={
43         "pk": f"TENANT#{tenant_id}#QUEJA#{queja_id}",
44         "sk": "META",
45         "tenantId": tenant_id,
46         "quejaId": queja_id,
47         "titulo": body.titulo,
48         "descripcion": body.descripcion,
49         "categoriaDeclarada": body.categoria_declarada.value,
50         "status": "EN_COLA",
51         "anonima": body.anonima,
52         "sede": body.sede,
53         "facultad": body.facultad,
54         "createdAt": now.isoformat(),
55         "ttl": int(now.timestamp()) + (90 * 24 * 60 * 60),
56     })
57
58     # 4. Encolar para procesamiento async
59     boto3.client("sqs").send_message(
60         QueueUrl=settings.process_queue_url,
61         MessageBody=json.dumps({
62             "eventType": "queja.creada",
63             "payload": {"quejaId": queja_id, "input": body.model_dump(
64                 mode="json")},
65             "metadata": {"correlationId": correlation_id},
66         }),
67         MessageAttributes={
68             "correlationId": {"DataType": "String", "StringValue":
69                 correlation_id},

```

```

68     },
69 )
70
71 logger.info("Queja creada exitosamente")
72 return accepted({
73     "quejaId": queja_id,
74     "status": "EN_COLA",
75     "correlationId": correlation_id,
76     "createdAt": now.isoformat(),
77     "estimatedAnalysisTime": 30,
78 }, correlation_id)

```

3.4 1.4 SAM Template (template.yaml)

Listing 24: src/infra/template.yaml

```

1  AWSTemplateFormatVersion: '2010-09-09'
2  Transform: AWS::Serverless-2016-10-31
3
4  Globals:
5    Function:
6      Runtime: python3.12
7      MemorySize: 512
8      Timeout: 30
9      Tracing: Active
10     Environment:
11       Variables:
12         POWERTOOLS_SERVICE_NAME: sentinel-academia
13         LOG_LEVEL: INFO
14         DYNAMODB_TABLE: !Ref QuejasTable
15         DEFAULT_TENANT_ID: universidad-ejemplo
16         OCI_COMPARTMENT_ID: !Ref OciCompartmentId
17         AWS_NODEJS_CONNECTION_REUSE_ENABLED: '1'
18
19  Parameters:
20    OciCompartmentId:
21      Type: String
22      Description: OCI Compartment OCID for Generative AI
23
24  Resources:
25
26    # DynamoDB Table
27    QuejasTable:
28      Type: AWS::DynamoDB::Table
29      Properties:
30        TableName: quejas
31        BillingMode: PAY_PER_REQUEST
32        AttributeDefinitions:
33          - { AttributeName: pk, AttributeType: S }
34          - { AttributeName: sk, AttributeType: S }
35          - { AttributeName: quejaId, AttributeType: S }
36          - { AttributeName: tenantId, AttributeType: S }
37          - { AttributeName: createdAt, AttributeType: S }
38          - { AttributeName: criticidad, AttributeType: S }
39        KeySchema:
40          - { AttributeName: pk, KeyType: HASH }
41          - { AttributeName: sk, KeyType: RANGE }

```



```

42     GlobalSecondaryIndexes:
43         - IndexName: GSI_ById
44           KeySchema: [{ AttributeName: quejaId, KeyType: HASH }]
45           Projection: { ProjectionType: ALL }
46         - IndexName: GSI_ByTenant
47           KeySchema:
48             - { AttributeName: tenantId, KeyType: HASH }
49             - { AttributeName: createdAt, KeyType: RANGE }
50           Projection: { ProjectionType: ALL }
51     TimeToLiveSpecification:
52       AttributeName: ttl
53       Enabled: true
54
55 # SQS Queues
56 ComplaintsQueue:
57   Type: AWS::SQS::Queue
58   Properties:
59     QueueName: complaints-queue
60     VisibilityTimeout: 180
61     MessageRetentionPeriod: 1209600
62     RedrivePolicy:
63       deadLetterTargetArn: !GetAtt ComplaintsDLQ.Arn
64       maxReceiveCount: 3
65
66 ComplaintsDLQ:
67   Type: AWS::SQS::Queue
68   Properties:
69     QueueName: complaints-dlq
70     MessageRetentionPeriod: 1209600
71
72 # SNS Topic
73 CriticalAlertsTopic:
74   Type: AWS::SNS::Topic
75   Properties:
76     TopicName: critical-alerts
77
78 # S3 Bucket
79 AdjuntosBucket:
80   Type: AWS::S3::Bucket
81   Properties:
82     CorsConfiguration:
83       CorsRules:
84         - AllowedOrigins: ['*']
85           AllowedMethods: [GET, PUT, POST]
86           AllowedHeaders: ['*']
87
88 # API Gateway
89 HttpApi:
90   Type: AWS::ApiGatewayV2::Api
91   Properties:
92     Name: sentinel-academia-api
93     ProtocolType: HTTP
94     CorsConfiguration:
95       AllowOrigins: ['*']
96       AllowMethods: ['*']
97       AllowHeaders: ['*']
98
99 # Lambda Functions

```

```

100 CreateQuejaFunction:
101     Type: AWS::Serverless::Function
102     Properties:
103         CodeUri: ../
104         Handler: handlers/create_queja.handler
105         Events:
106             Api:
107                 Type: HttpApi
108                 Properties:
109                     ApiId: !Ref HttpApi
110                     Path: /quejas
111                     Method: POST
112             Policies:
113                 - DynamoDBCrudPolicy: { TableName: !Ref QuejasTable }
114                 - SQSSendMessagePolicy: { QueueName: !GetAtt ComplaintsQueue.
115                                         QueueName }
116
117 GetQuejaFunction:
118     Type: AWS::Serverless::Function
119     Properties:
120         CodeUri: ../
121         Handler: handlers/get_queja.handler
122         Events:
123             Api:
124                 Type: HttpApi
125                 Properties:
126                     ApiId: !Ref HttpApi
127                     Path: /quejas/{id}
128                     Method: GET
129             Policies:
130                 - DynamoDBReadPolicy: { TableName: !Ref QuejasTable }
131
132 ListQuejasFunction:
133     Type: AWS::Serverless::Function
134     Properties:
135         CodeUri: ../
136         Handler: handlers/list_quejas.handler
137         Events:
138             Api:
139                 Type: HttpApi
140                 Properties:
141                     ApiId: !Ref HttpApi
142                     Path: /quejas
143                     Method: GET
144             Policies:
145                 - DynamoDBReadPolicy: { TableName: !Ref QuejasTable }
146
147 ProcessQuejaFunction:
148     Type: AWS::Serverless::Function
149     Properties:
150         CodeUri: ../
151         Handler: handlers/process_queja.handler
152         MemorySize: 1024
153         Timeout: 300
154         Events:
155             SQS:
156                 Type: SQS
157                 Properties:

```

```

157         Queue: !GetAtt ComplaintsQueue.Arn
158         BatchSize: 5
159     Policies:
160         - DynamoDBCrudPolicy: { TableName: !Ref QuejasTable }
161         - S3ReadPolicy: { BucketName: !Ref AdjuntosBucket }
162
163 Outputs:
164     ApiUrl:
165         Value: !GetAtt HttpApi.ApiEndpoint
166         Export: { Name: SentinelApiUrl }
167     TableName:
168         Value: !Ref QuejasTable
169     QueueUrl:
170         Value: !Ref ComplaintsQueue

```

3.5 1.5 Deploy del backend

Listing 25: Build y deploy con SAM

```

1  cd src/infra
2
3  # Build
4  sam build
5  # Output: Build Succeeded
6
7  # Deploy (primera vez)
8  sam deploy --guided
9  # Responder:
10 #   Stack Name: sentinel-academia-dev
11 #   AWS Region: us-east-1
12 #   Parameter OciCompartmentId: <tu OCID>
13 #   Confirmar cambios: y
14 #   Permitir IAM: y
15 #   Save to samconfig.toml: y
16
17 # Deploys subsecuentes
18 sam deploy
19
20 # Ver outputs
21 aws cloudformation describe-stacks \
22     --stack-name sentinel-academia-dev \
23     --query 'Stacks[0].Outputs'

```

4 FASE 2: Backend LLM (OCI Integration)

4.1 2.1 Cliente OCI Generative AI

Listing 26: src/services/oci_genai.py

```

1  """Cliente para OCI Generative AI Service."""
2  from typing import Any
3  import oci
4  from oci.generative_ai_inference import GenerativeAiInferenceClient
5  from oci.generative_ai_inference.models import (
6      GenerateTextDetails, ChatDetails, TextContent
7  )

```

```

8
9
10 class OCIGenAIClient:
11     def __init__(self, compartment_id: str, region: str = "us-chicago-1"):
12         config = oci.config.from_file("~/oci/config", "DEFAULT")
13         self.client = GenerativeAiInferenceClient(config=config, region=region)
14         self.compartment_id = compartment_id
15
16     def generate_text(
17         self, prompt: str, model_id: str = "cohere.command-r",
18         max_tokens: int = 1000, temperature: float = 0.3
19     ) -> str:
20         details = GenerateTextDetails(
21             compartment_id=self.compartment_id,
22             serving_mode={"modelId": model_id, "servingType": "ON_DEMAND"},
23             inference_request={
24                 "prompt": prompt,
25                 "maxTokens": max_tokens,
26                 "temperature": temperature,
27                 "isStream": False,
28             },
29         )
30         response = self.client.generate_text(generate_text_details=details)
31         return response.data.inference_response.inference_text

```

4.2 2.2 LLM Router con Circuit Breaker

Listing 27: src/services/llm_router.py

```

1  """Router de LLM con fallback chain."""
2  import opossum
3  from .oci_genai import OCIGenAIClient
4
5  # Breakers por modelo
6  breaker_primary = opossum.CircuitBreaker(
7      call=lambda p: oci.generate(p, "cohere.command-r"),
8      timeout=30,
9      error_threshold_percentage=50,
10     reset_timeout=30,
11 )
12
13 breaker_secondary = opossum.CircuitBreaker(
14     call=lambda p: oci.generate(p, "meta.llama-3.1-70b-instruct"),
15     timeout=30,
16     error_threshold_percentage=50,
17     reset_timeout=30,
18 )
19
20
21 def generate_with_fallback(prompt: str) -> str:
22     """Intenta modelos en orden, con fallback."""
23     models = [
24         ("cohere.command-r", breaker_primary),

```

```

25         ("meta.llama-3.1-70b-instruct", breaker_secondary),
26     ]
27
28     for model_name, breaker in models:
29         try:
30             return breaker.call(prompt)
31         except Exception:
32             continue
33
34     return json.dumps({"error": "Todos los modelos fallaron"})

```

5 FASE 3: Frontend Vue 3

5.1 3.1 Setup

Listing 28: Setup del frontend

```

1  cd web
2  npm install
3  cp .env.example .env.local
4  # Editar .env.local con la URL del API Gateway
5
6  npm run dev
7  # Abrir http://localhost:5173

```

5.2 3.2 Deploy a Vercel

Listing 29: Deploy a Vercel con CLI

```

1  npm install -g vercel
2
3  # Login
4  vercel login
5
6  # Deploy
7  vercel --prod
8
9  # Opciones:
10 # - Framework: Vite
11 # - Build: npm run build
12 # - Output: dist
13 # - Root: ./

```

6 FASE 4: Archivos + Email

6.1 4.1 S3 + Presigned URLs

Listing 30: src/handlers/get_upload_url.py

```

1  """POST /api/uploads/presigned - Genera presigned URL para S3."""
2  import uuid
3  from typing import Any
4  import boto3
5  from aws_lambda_powertools import Logger

```

```

6 from botocore.config import Config
7 from ._shared.config import settings
8 from ._shared.correlation_id import get_correlation_id
9 from ._shared.errors import with_error_handling
10 from ._shared.http_response import ok
11
12 logger = Logger(service="get-upload-url")
13
14
15 @logger.inject_lambda_context
16 @with_error_handling
17 def handler(event: dict, context: Any) -> dict:
18     correlation_id = get_correlation_id(event)
19     body = json.loads(event.get("body", "{}"))
20     filename = body["filename"]
21     content_type = body["contentType"]
22     queja_id = body["quejaId"]
23
24     s3 = boto3.client("s3", config=Config(signature_version="s3v4"))
25     key = f"tenants/{settings.default_tenant_id}/quejas/{queja_id}/{uuid
        .uuid4()}-{filename}"
26
27     url = s3.generate_presigned_url(
28         "put_object",
29         Params={"Bucket": settings.s3_bucket, "Key": key, "ContentType":
            content_type},
30         ExpiresIn=300,
31     )
32
33     return ok({"uploadUrl": url, "key": key, "expiresIn": 300},
        correlation_id)

```

6.2 4.2 Email con SES

Listing 31: src/handlers/notify.py

```

1 """Consumer SNS que envia emails via SES para casos CRITICOS."""
2 import json
3 from typing import Any
4 import boto3
5 from aws_lambda_powertools import Logger
6 from ._shared.config import settings
7 from ._shared.errors import with_error_handling
8
9 logger = Logger(service="notify")
10
11
12 @logger.inject_lambda_context
13 @with_error_handling
14 def handler(event: dict, context: Any) -> None:
15     for record in event.get("Records", []):
16         sns = record["Sns"]
17         message = json.loads(sns["Message"])
18         analysis = message.get("detail", {}).get("analysis", {})
19
20         if analysis.get("criticidad") != "CRITICA":
21             continue

```

```

22     ses = boto3.client("ses")
23     ses.send_email(
24         Source=settings.ses_from_email,
25         Destination={"ToAddresses": [settings.bienestar_email,
26                                     settings.director_email]},
27         Message={
28             "Subject": {"Data": f"ALERTA: Queja CRITICA {message['
29                             detail']['quejaId']}"},
30             "Body": {
31                 "Text": {"Data": f"Queja critica detectada.\nAccion:
32                             {analysis.get('accionSugerida', 'N/A')}}"}
33         },
34     )
35     logger.info("Email sent")

```

7 FASE 5: RAG (Opcional)

7.1 5.1 Ingesta del reglamento

Listing 32: src/services/rag_ingest.py

```

1  """Ingesta de documentos para RAG."""
2  import boto3
3  import cohere
4  from .oci_genai import OCIGenAIClient
5
6
7  def ingest_document(s3_key: str, oci_client: OCIGenAIClient):
8      """Descarga documento de S3, lo chunk-ear, genera embeddings, guarda
9      en DynamoDB."""
10     s3 = boto3.client("s3")
11     obj = s3.get_object(Bucket="reglamento-bucket", Key=s3_key)
12     text = obj["Body"].read().decode("utf-8")
13
14     # Chunking (500 chars con overlap de 50)
15     chunks = [text[i:i+500] for i in range(0, len(text), 450)]
16
17     # Generar embeddings con Cohere
18     co = cohere.Client("tu-api-key")
19     response = co.embed(texts=chunks, model="embed-multilingual-v3.0")
20     embeddings = response.embeddings
21
22     # Guardar en DynamoDB
23     # ... (similar a otros repos)

```

8 FASE 6: Testing

8.1 6.1 Tests unitarios

Listing 33: Ejecutar tests

```

1  cd src
2  source .venv/bin/activate

```

```

3
4 # Todos los tests
5 pytest
6
7 # Con coverage
8 pytest --cov=. --cov-report=html
9
10 # Solo un test específico
11 pytest tests/handlers/test_create_queja.py
12
13 # Ver reporte HTML
14 open htmlcov/index.html

```

Listing 34: tests/handlers/test_create_queja.py

```

1 import json
2 import pytest
3 from moto import mock_aws
4 import boto3
5 from src.handlers.create_queja import handler
6
7
8 @pytest.fixture
9 def event():
10     return {
11         "body": json.dumps({
12             "titulo": "Test",
13             "descripcion": "Descripcion suficientemente larga",
14             "categoriaDeclarada": "ACADEMICA",
15         }),
16         "headers": {"x-correlation-id": "test-123"},
17     }
18
19
20 @mock_aws
21 def test_create_queja_success(event, monkeypatch):
22     # Setup
23     client = boto3.client("dynamodb", region_name="us-east-1")
24     client.create_table(
25         TableName="quejas",
26         KeySchema=[{"AttributeName": "pk", "KeyType": "HASH"}],
27         AttributeDefinitions=[{"AttributeName": "pk", "AttributeType": "S"}],
28         BillingMode="PAY_PER_REQUEST",
29     )
30     monkeypatch.setenv("DYNAMODB_TABLE", "quejas")
31
32     # Mock SQS
33     sqs = boto3.client("sqs", region_name="us-east-1")
34     q = sqs.create_queue(QueueName="test-queue")
35     monkeypatch.setenv("PROCESS_QUEUE_URL", q["QueueUrl"])
36
37     # Act
38     result = handler(event, None)
39
40     # Assert
41     assert result["statusCode"] == 202
42     body = json.loads(result["body"])
43     assert "quejaId" in body

```


9 FASE 7: Deploy Final

9.1 7.1 Deploy completo

Listing 35: Deploy final paso a paso

```
1 # 1. Backend
2 cd src/infra
3 sam build && sam deploy
4
5 # Guardar la URL del API
6 API_URL=$(aws cloudformation describe-stacks --stack-name sentinel-
   academia-dev --query 'Stacks[0].Outputs[?OutputKey=='ApiUrl'].
   OutputValue' --output text)
7 echo "API URL: $API_URL"
8
9 # 2. Frontend
10 cd ../../web
11 # Actualizar VITE_API_URL en .env
12 echo "VITE_API_URL=$API_URL" > .env.production
13 npm run build
14 vercel --prod
15
16 # 3. Verificacion
17 curl $API_URL/quejas
18 # Debe responder 200
```

9.2 7.2 Smoke tests

Listing 36: Verificación end-to-end

```
1 # Crear queja de prueba
2 curl -X POST $API_URL/quejas \
3   -H "Content-Type: application/json" \
4   -H "x-correlation-id: smoke-test-1" \
5   -d '{
6     "titulo": "Smoke test",
7     "descripcion": "Verificación de deploy",
8     "categoriaDeclarada": "OTRA"
9   }'
10
11 # Ver logs
12 sam logs -n CreateQuejaFunction --tail
13
14 # Verificar que se guardó
15 aws dynamodb scan --table-name quejas --max-items 1
```

10 Apéndice A: Comandos útiles

Listing 37: Cheatsheet de comandos

```
1 # Ver logs en tiempo real
```

```

2 sam logs -n CreateQuejaFunction --tail --stack-name sentinel-academia-
  dev
3
4 # Invocar una Lambda manualmente
5 sam remote invoke -n CreateQuejaFunction --event '{"body":"{}","headers
  ":{}}'
6
7 # Ver outputs del stack
8 aws cloudformation describe-stacks --stack-name sentinel-academia-dev --
  query 'Stacks[0].Outputs'
9
10 # Borrar todo (empezar de cero)
11 sam delete --stack-name sentinel-academia-dev
12
13 # Ver costos del mes
14 aws ce get-cost-and-usage --time-period Start=2025-01-01,End=2025-01-31
  --granularity MONTHLY --metrics BlendedCost
15
16 # Test del LLM con OCI
17 oci generative-ai-inference generate-text \
18   --compartment-id $OCI_COMPARTMENT_ID \
19   --serving-mode '{"modelId":"cohere.command-r","servingType":"ON_DEMAND
  "}' \
20   --inference-request '{"prompt":"Test","maxTokens":50}'

```

11 Apéndice B: Troubleshooting

LabRole does not exist

Solución: En AWS Academy, el LabRole es creado automáticamente. Si no existe, ir a IAM → Roles → Create role → AWS service → Lambda → Attach policies: AWSLambdaBasicExecutionRole y AWSLambdaVPCLAccessExecutionRole.

OCI 404 Not Found en GenAI

Solución:

1. Verificar que el Compartment OCID es correcto
2. Verificar que la región soporta GenAI (us-chicago-1, eu-frankfurt-1)
3. Verificar policies del compartment

CORS error en frontend

Solución: Verificar que el API Gateway tiene CORS configurado:

```

1 CorsConfiguration:
2   AllowOrigins: ['*']
3   AllowMethods: ['*']
4   AllowHeaders: ['*']

```

Lambda timeout

Solución: En `template.yaml`, subir el Timeout:

```
1 Globals:
2   Function:
3     Timeout: 120 # Subir de 30 a 120 segundos
```

12 Apéndice C: Variables de entorno

Variable	Default	Propósito
POWERTOOLS_SERVICE_NAME	sentinel-academia	Nombre del servicio en logs
LOG_LEVEL	INFO	Nivel de logging
AWS_REGION	us-east-1	Region de AWS
DYNAMODB_TABLE	quejas	Nombre de la tabla
S3_BUCKET	-	Bucket de adjuntos
PROCESS_QUEUE_URL	-	URL de SQS
CRITICAL_ALERTS_TOPIC_ARN	-	ARN del topic SNS
OCI_COMPARTMENT_ID	-	OCID del compartment
OCI_REGION	us-chicago-1	Region de OCI
SES_FROM_EMAIL	-	Email remitente
BIENESTAR_EMAIL	-	Email de bienestar
DIRECTOR_EMAIL	-	Email del director
DEFAULT_TENANT_ID	universidad-ejemplo	Tenant por defecto

13 Apéndice D: Costos esperados

Servicio	Hackaton (4 días)	Producción (mes)
AWS Lambda	\$0.00	\$0.01
API Gateway	\$0.00	\$0.03
DynamoDB	\$0.05	\$3.00
SQS/SNS/EventBridge	\$0.00	\$0.05
S3	\$0.00	\$0.30
CloudWatch	\$0.10	\$0.50
Subtotal AWS	\$0.15	\$4.00
OCI Cohere R	\$0.50	\$5.00
OCI Llama 3.1	\$0.10	\$2.00
OCI Document AI	\$0.00	\$5.00
Subtotal OCI	\$0.60	\$12.00
Vercel	\$0.00	\$0.00
TOTAL	\$0.75	\$16.00